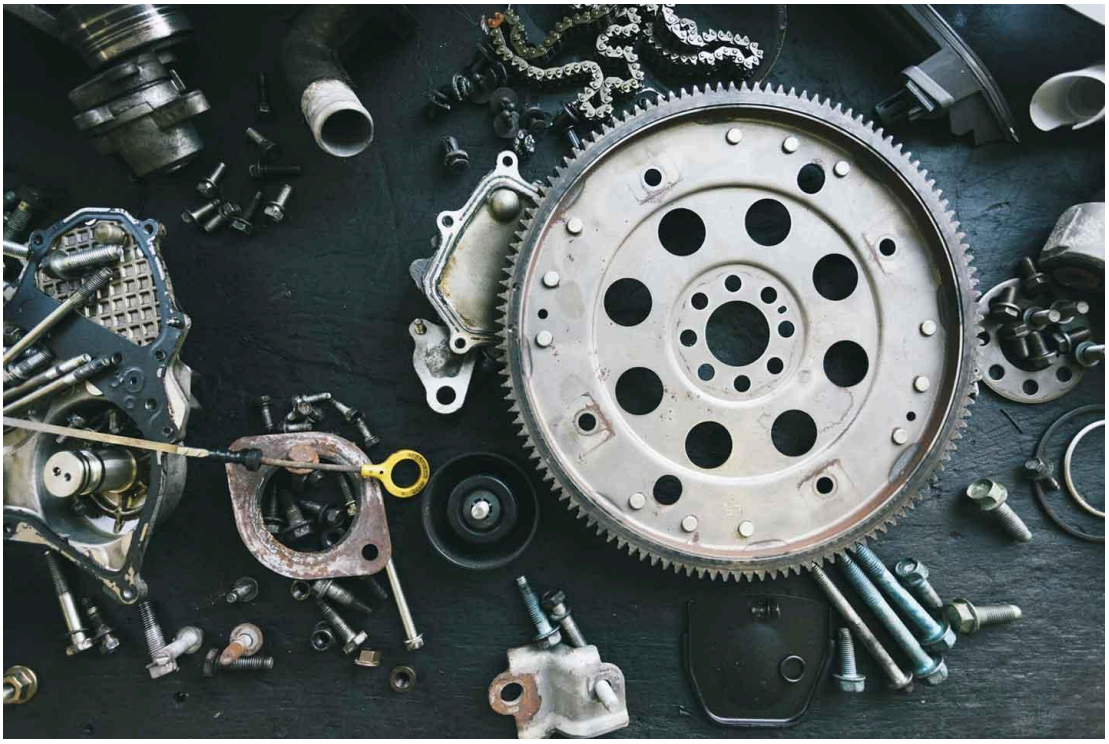
Tema 2

Persistencia con JPA y Mapeo de Entidades

Digitech
FP OFICIAL DIGITAL & TECH



Francisco Belda

ÍNDICE

Teoría. Fundamentos de ORM y JPA **3**
El problema de la impedancia placa-objeto 3
JPA y Hibernate 3
Paso a Paso: Configuración de la Base de Datos **4**
Añadir dependencias en el pom.xml 4
Configuración en application.properties 5
Explicación de propiedades clave 5
Archivo Config 6
Práctica: Mapeo de Entidades con JPA **7**
Creación de la clase Entidad 7
Desglose de Anotaciones JPA utilizadas 9
Ejercicios **10**
Conceptos Clave Recordados 10
Ejercicios Propuestos para el Alumno 10

Teoría. Fundamentos de ORM y JPA

El problema de la impedancia placa-objeto

Tradicionalmente, trabajar con bases de datos relacionales desde lenguajes orientados a objetos como Java implicaba escribir código repetitivo (JDBC plano) para mapear filas y columnas de tablas SQL a objetos y atributos en memoria. Este desfase conceptual se conoce como impedance mismatch.

Los ORM (Object-Relational Mapping) solucionan esto permitiendo manipular los datos de la base de datos directamente como si fueran objetos y colecciones de Java, encargándose el framework de traducir las operaciones a consultas SQL por debajo.

JPA y Hibernate

JPA (Java Persistence API): Es una especificación estándar de Java (un conjunto de interfaces y reglas) que define cómo persistir datos entre objetos Java y una base de datos relacional.

Hibernate: Es el framework ORM más popular que implementa JPA. Spring Boot encapsula Hibernate a través del starter de Spring Data JPA, facilitando enormemente su configuración.

Paso a Paso: Configuración de la Base de Datos

Añadir dependencias en el pom.xml

Para trabajar con bases de datos y JPA, necesitamos incorporar dos dependencias fundamentales en nuestro archivo **pom.xml**:

- **Spring Data JPA**: Facilita la creación de la capa de persistencia abstrayendo las consultas SQL.
- **H2 Database**: Una base de datos ultrarrápida ligera que se ejecuta completamente en memoria RAM (ideal para pruebas y desarrollo educativo inicial sin necesidad de instalar servidores externos). Opcionalmente, se puede configurar MySQL.

```
<dependencies>
    <!-- Starter para JPA y Hibernate -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- Base de datos H2 en memoria -->
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-h2console</artifactId>
    </dependency>
```

```
<!-- Starter Web (ya incluido en sesión anterior) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

CONFIGURACIÓN EN APPLICATION.PROPERTIES

Modifica el archivo ubicado en src/main/resources/application.properties para configurar la conexión a H2 y habilitar la consola web de administración:

```
# Configuración de la conexión a la base de datos H2 en
memoria
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Configuración de Hibernate / JPA
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Habilitar la consola web de H2 para visualizar la base de
datos
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Explicación de propiedades clave

spring.jpa.hibernate.ddl-auto=update: Ordena a Hibernate que cree o actualice automáticamente las tablas en la base de datos según nuestras clases entidad en Java.

spring.jpa.show-sql=true: Muestra por consola las sentencias SQL que Hibernate ejecuta internamente (muy útil para depurar).

spring.h2.console.enabled=true: Permite acceder a una interfaz gráfica web en http://localhost:8080/h2-console para consultar las tablas creadas.

Archivo Config

Y creamos un paquete **config** y un fichero de configuración **H2ConsoleConfig.java**:

```
import org.h2.server.web.JakartaWebServlet;

import org.springframework.boot.web.servlet.ServletRegistrationBean;

import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;


@Configuration

public class H2ConsoleConfig {


   @Bean

                public      ServletRegistrationBean<JakartaWebServlet>
h2servletRegistration() {

        ServletRegistrationBean<JakartaWebServlet> registration =

                                    new   ServletRegistrationBean<>(new
JakartaWebServlet());

        registration.addUrlMappings("/h2-console/*");

        return registration;

   }

}
```

Práctica: Mapeo de Entidades con JPA

Vamos a modelar una aplicación sencilla de gestión académica. Crearemos una entidad llamada Estudiante.

Creación de la clase Entidad

Crea un paquete llamado model o entity dentro de **com.educacion.demospring** y dentro de él la clase **Estudiante.java**:

```
package com.educacion.demospring.model;


import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;


@Entity
@Table(name = "estudiantes")
public class Estudiante {


    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;


    @Column(name = "nombre", nullable = false, length = 50)
    private String nombre;
```

```
    @Column(name = "correo", nullable = false, unique = true,
length = 100)
    private String correo;


    @Column(name = "edad")
    private int edad;


    // Constructor vacío obligatorio para JPA/Hibernate
    public Estudiante() {
    }
        // Constructor con parámetros (sin ID, ya que es
autoincremental)
    public Estudiante(String nombre, String correo, int edad)
{
        this.nombre = nombre;
        this.correo = correo;
        this.edad = edad;
    }


    // Getters y Setters
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
```

```
        this.nombre = nombre;
    }
    public String getCorreo() {
        return correo;
    }
    public void setCorreo(String correo) {
        this.correo = correo;
    }
    public int getEdad() {
        return edad;
    }
    public void setEdad(int edad) {
        this.edad = edad;
    }
}
```

Desglose de Anotaciones JPA utilizadas

- **`@Entity`**: Indica a Hibernate que esta clase Java debe ser tratada como una entidad persistente vinculada a una tabla de base de datos.
- **`@Table(name = "estudiantes")`**: Permite personalizar el nombre exacto de la tabla en SQL (por defecto usaría el nombre de la clase).
- **`@Id`**: Especifica que este atributo es la clave primaria (Primary Key) de la tabla.
- **`@GeneratedValue(strategy = GenerationType.IDENTITY)`**: Indica que el valor de la clave primaria se generará de manera autoincremental por la base de datos.
- **`@Column`**: Define restricciones a nivel de columna SQL (como nullable = false para campos obligatorios, unique = true para evitar duplicados, y longitudes máximas).

Ejercicios

Conceptos Clave Recordados

- El ORM resuelve el desfase entre objetos Java y tablas relacionales SQL.
- spring.jpa.hibernate.ddl-auto=update automatiza la generación de esquemas de base de datos en desarrollo.
- Toda entidad JPA requiere obligatoriamente una clave primaria (@Id) y un constructor vacío sin argumentos.

Ejercicios Propuestos para el Alumno

1. **Crear la Entidad Profesor**: Diseña una nueva entidad llamada Profesor dentro del paquete model con los siguientes atributos: id (Long, autoincremental), nombreCompleto (String, obligatorio), especialidad (String, ej. "Programación") y experienciaAnios (int). Configura las restricciones pertinentes con @Column.
2. **Verificación en H2 Console**: Arranca la aplicación con mvn spring-boot:run, accede a http://localhost:8080/h2-console y comprueba que Hibernate ha generado automáticamente las tablas estudiantes y profesores en la base de datos en memoria.